

Swapping

1 Core Idea

Definition

Swapping is a memory-management technique where the operating system moves an entire process between main memory and disk.

When RAM is not large enough to hold all active processes, the operating system keeps some processes in memory and stores idle processes on disk.

Main Purpose

Swapping lets the system run more processes than can fit in RAM at the same time.

2 Why Swapping Is Needed

Modern systems often start many processes automatically. Some check for updates, incoming mail, network connections, or background services. Large applications can also need hundreds of megabytes or many gigabytes.

Situation	Memory pressure
Many background processes	RAM is consumed before the user opens an application.
Large applications	Programs such as image editors may need hundreds of MB or more.
Limited physical memory	Not every process can stay in RAM permanently.

3 Two Approaches to Memory Overload

Approach	Main idea
Swapping	Move a whole process into memory, run it, then move the whole process back to disk.
Virtual memory	Run a process even when only part of it is in main memory.

This Note

Swapping works at the level of whole processes. Virtual memory works at a finer level and is studied separately.

4 How Swapping Works

The basic cycle is simple:

1. Bring a process from disk into RAM.
2. Run it for a while.
3. If memory is needed, move an idle process back to disk.
4. Load another process into the freed memory.

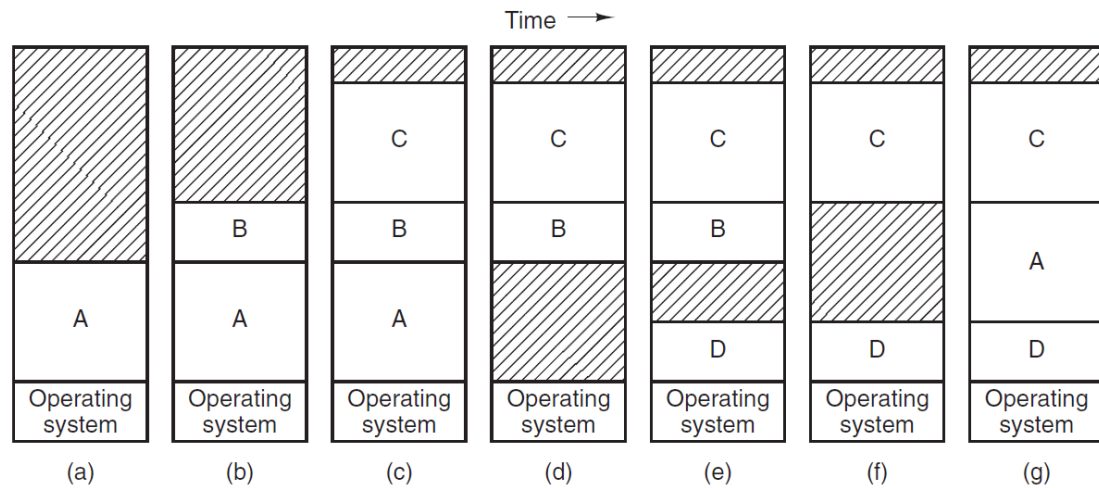


Figure 3-4. Memory allocation changes as processes come into memory and leave it. The shaded regions are unused memory.

Stage	Meaning
(a)	Only process <i>A</i> is in memory.
(b), (c)	Processes <i>B</i> and <i>C</i> are brought into memory.
(d)	Process <i>A</i> is swapped out, leaving a hole.
(e), (f)	Process <i>D</i> is loaded; later <i>B</i> is swapped out.
(g)	Process <i>A</i> returns, possibly at a different memory location.

5 Relocation After Swapping

When a process is swapped back in, it may not return to the same physical address.

Important Consequence

If a process moves to a different physical location, its addresses must still work correctly.

This can be handled in two ways:

Method	Explanation
Software relocation	Adjust addresses when the process is loaded.
Hardware relocation	Use base and limit registers during execution.

Why Base and Limit Help

The base register can point to the process's current physical starting address. If the process moves, the operating system updates the base register.

6 Holes and Memory Compaction

Swapping creates **holes**: unused regions between processes.

Memory Compaction

Memory compaction moves processes downward in memory so many small holes become one large hole.

Idea	Cost
Move processes together	Creates one large free region.
Copy large parts of memory	Uses a lot of CPU time and memory bandwidth.

Why Compaction Is Rare

Compaction can be expensive. Copying many gigabytes of RAM takes noticeable time, so systems avoid doing it frequently.

7 Allocating Space for Growth

If process size never changes, allocation is easy: give the process exactly the memory it needs.

Real processes often grow:

- the heap grows when the process dynamically allocates memory;
- the stack grows when function calls, local variables, and return addresses are added.

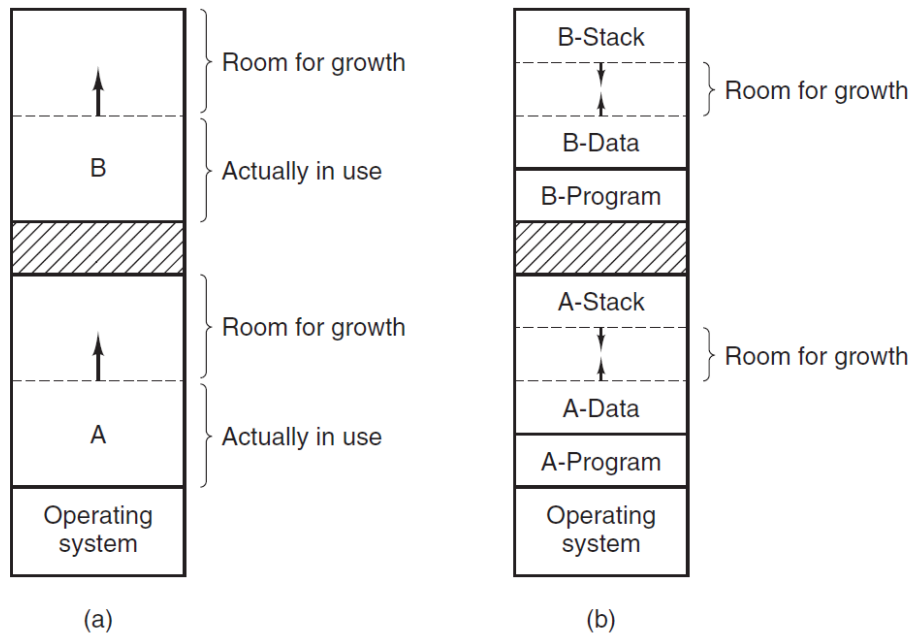


Figure 3-5. (a) Allocating space for a growing data segment. (b) Allocating space for a growing stack and a growing data segment.

8 Growing Data Segment

In Fig. 3-5(a), extra space is reserved above each process.

Part	Meaning
Actually in use	Memory currently needed by the process.
Room for growth	Extra space reserved so the process can grow without being moved immediately.
Hole between processes	Unused memory that may be allocated later.

Benefit

Allocating extra room reduces how often a growing process must be moved or swapped out.

9 Growing Stack and Heap

In Fig. 3-5(b), each process has:

- program text at the bottom;
- data or heap above the program text, growing upward;
- stack at the top, growing downward;
- free space between heap and stack.

Growth Direction

heap grows upward and stack grows downward

The free space between them can be used by either segment. If the space runs out, the operating system must take action.

10 What Happens When a Process Cannot Grow?

Case	OS action
Adjacent hole exists	Allow the process to grow into the hole.
No adjacent hole	Move the process to a larger hole, if one exists.
No large enough hole	Swap out one or more processes to create space.
No RAM and swap space full	Suspend the process until space is available, or kill it.

Do Not Swap Extra Space

When a process is swapped to disk, only memory actually in use should be written out. Swapping unused growth space wastes disk I/O.

11 Swapping Summary

Point	Takeaway
Unit moved	Whole process.
Main benefit	More processes can exist than fit in RAM at once.
Main cost	Disk I/O is slow.
Main side effect	Holes appear in memory.
Compaction	Combines holes, but costs CPU time.
Growth space	Reduces movement, but unused space should not be swapped.